

GiftKeeper

Advanced Automated Software Testing Project

Comprehensive Academic Report

Table of Contents

1. Introduction
2. Project Motivation
3. Project Objectives
4. Technologies and Tools
5. Software Architecture
6. Multi-Module Maven Structure
7. Domain Model Explanation
8. Application Layer
9. Persistence Layer
10. GUI and CLI Interfaces
11. Test-Driven Development Workflow
12. Testing Strategy
13. Unit Testing
14. Integration Testing
15. GUI Testing
16. End-to-End Testing
17. Mocking with Mockito
18. Mutation Testing with PIT
19. Code Coverage with JaCoCo
20. Continuous Integration with GitHub Actions
21. Code Quality Analysis with SonarCloud
22. Docker and Testcontainers
23. Conclusion
24. Screenshots

1. Introduction

GiftKeeper is a modular Java application developed as a complete academic project for the Automated Software Testing course.

The goal of the project is not only to implement a functional application, but also to demonstrate a complete modern software engineering workflow based on:

- Test-Driven Development (TDD)
- Automated testing
- Continuous Integration
- Mutation testing
- Build automation
- Code quality analysis
- Docker-based reproducible environments
- Modular software architecture

The application allows users to manage people, occasions, gift ideas, budgets, and gift history using both graphical and command-line interfaces.

The project was intentionally designed to reflect real-world software engineering practices taught during the course.

2. Project Motivation

The project was created to demonstrate how automated testing techniques can guide the development of reliable and maintainable software systems.

Instead of writing tests after implementation, the project follows a testing-first approach where tests define expected behavior before production code is written.

This approach improves:

- reliability
- maintainability
- modularity
- refactoring safety
- code quality

The project also demonstrates how modern development pipelines integrate testing, quality analysis, CI/CD, and containerization into a unified workflow.

3. Project Objectives

The main objectives of the project are:

- Apply Test-Driven Development principles
 - Implement clean modular architecture
 - Create isolated and reproducible tests
 - Demonstrate automated build pipelines
 - Use mutation testing to validate test quality
 - Use static analysis tools to improve maintainability
 - Integrate Docker and Testcontainers
 - Implement both GUI and CLI interfaces
 - Use GitHub Actions for continuous integration
 - Apply professional Java development practices
-

4. Technologies and Tools

Technology	Purpose
Java 17	Main programming language
Maven	Build automation
JUnit 5	Unit testing
AssertJ	Fluent assertions
Mockito	Mocking dependencies
PIT	Mutation testing
JaCoCo	Code coverage
SonarCloud	Code quality analysis
Git/GitHub	Version control
GitHub Actions	Continuous Integration
PostgreSQL	Database
Hibernate/JPA	Persistence layer
Testcontainers	Containerized integration tests
Docker	Reproducible environments

Technology	Purpose
Swing	GUI interface

5. Software Architecture

The project follows a layered modular architecture.

The system is divided into independent Maven modules in order to improve maintainability, separation of concerns, and testability.

The architecture separates:

- domain logic
- persistence
- application services
- user interfaces
- testing infrastructure

This design allows components to evolve independently while remaining loosely coupled.

6. Multi-Module Maven Structure

The project contains the following modules:

```
giftkeeper-parent
├── giftkeeper-domain
├── giftkeeper-app
├── giftkeeper-persistence-api
├── giftkeeper-persistence-jpa
├── giftkeeper-cli
├── giftkeeper-gui
├── giftkeeper-bdd
└── giftkeeper-e2e
```

Each module has a dedicated responsibility.

7. Domain Model Explanation

The domain layer represents the business entities of the application.

Important entities include:

- Person
- Occasion
- GiftIdea
- GiftHistoryEntry
- Budget
- ReminderPolicy

The domain layer contains:

- validation logic
- immutable objects
- business constraints
- state transitions

The domain layer is intentionally independent from databases and frameworks.

This improves:

- portability
 - maintainability
 - testability
-

8. Application Layer

The application layer coordinates business operations between the domain and persistence layers.

Main responsibilities:

- create persons
- create occasions
- manage gift ideas
- update gift status
- retrieve information

The service layer acts as the central orchestration layer of the application.

9. Persistence Layer

The persistence layer uses:

- PostgreSQL
- Hibernate ORM
- JPA repositories

The architecture separates persistence interfaces from concrete implementations.

This abstraction allows easier testing and improves modularity.

10. GUI and CLI Interfaces

The application provides:

- graphical desktop interface
- command-line interface

The GUI demonstrates functional desktop application testing concepts.

The CLI demonstrates lightweight interaction and logging.

Both interfaces share the same application layer.

11. Test-Driven Development Workflow

The project follows the Red-Green-Refactor cycle:

1. Write failing tests
2. Implement minimal production logic
3. Refactor safely while tests remain green

Tests are treated as executable specifications of system behavior.

12. Testing Strategy

The project implements multiple testing layers:

- Unit Testing
- Integration Testing
- GUI Testing
- End-to-End Testing
- Mutation Testing
- Static Code Analysis

This layered strategy improves confidence in system correctness.

13. Unit Testing

Unit tests verify isolated components individually.

The project contains extensive unit tests for:

- domain entities
- services
- validation logic
- repository behavior

JUnit 5 and AssertJ are heavily used.

14. Integration Testing

Integration tests validate interactions between:

- application services
- repositories
- PostgreSQL database

Real database containers are created automatically using Testcontainers.

This guarantees realistic testing environments.

15. GUI Testing

GUI tests validate the graphical interface behavior.

The project verifies:

- frame creation
- GUI initialization
- interaction behavior

GUI testing demonstrates functional testing concepts discussed during the course.

16. End-to-End Testing

End-to-end tests simulate complete application workflows.

These tests verify:

- database interaction
- service integration
- persistence behavior
- application consistency

The tests provide high-level verification of the complete system.

17. Mocking with Mockito

Mockito is used to isolate dependencies during testing.

Mocking allows:

- isolation of components
- deterministic behavior
- removal of external dependencies

This improves unit test reliability and execution speed.

18. Mutation Testing with PIT

PIT mutation testing validates the effectiveness of tests.

Instead of measuring only executed lines, PIT introduces artificial faults (mutations) into the code.

Tests must detect these mutations.

This provides stronger confidence in test quality than simple coverage metrics.

19. Code Coverage with JaCoCo

JaCoCo measures:

- line coverage
- branch coverage

The project achieved very high coverage percentages across core modules.

Coverage reports were integrated into CI pipelines and SonarCloud analysis.

20. Continuous Integration with GitHub Actions

GitHub Actions automatically executes:

- compilation
- unit tests
- integration tests
- mutation testing
- SonarCloud analysis

after every push.

This ensures continuous verification of software quality.

21. Code Quality Analysis with SonarCloud

SonarCloud analyzes:

- maintainability
- duplicated code
- smells
- reliability
- coverage

The project successfully passes the Quality Gate.

Issues detected by SonarCloud were corrected during development.

22. Docker and Testcontainers

Docker and Testcontainers were used to provide isolated PostgreSQL environments.

Benefits include:

- reproducibility
- portability
- clean environments
- realistic integration testing

This eliminates dependency on manually configured local databases.

23. Conclusion

GiftKeeper demonstrates a complete automated software testing workflow using modern Java engineering practices.

The project combines:

- TDD
- automated testing
- CI/CD
- mutation testing
- Docker

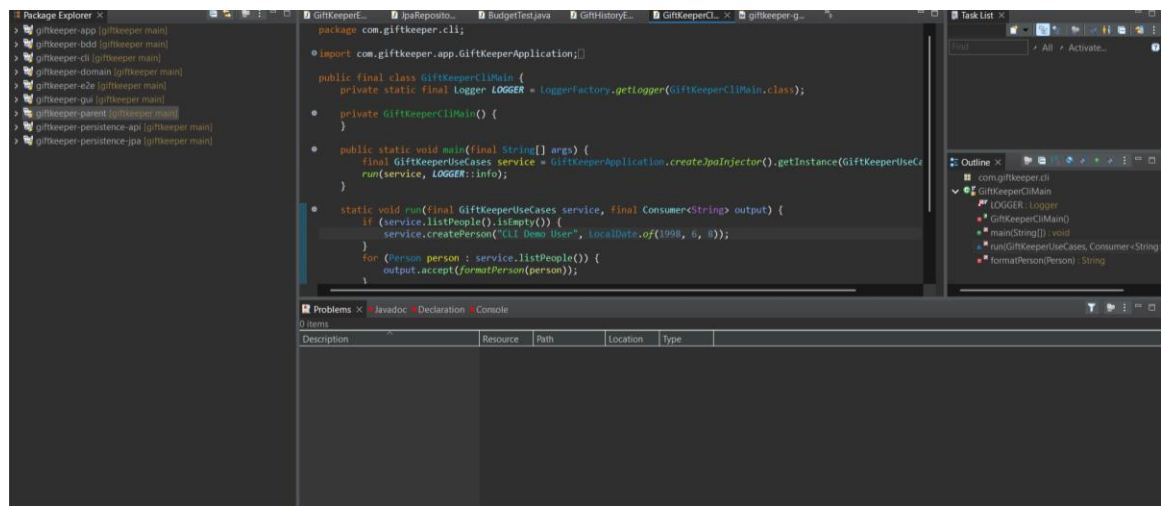
- static analysis
- modular architecture

into a realistic academic software engineering project.

The final result demonstrates both practical implementation skills and theoretical understanding of automated software testing methodologies.

24. Screenshots

- Eclipse workspace



- Maven BUILD SUCCESS

```
[INFO] -----
[INFO] Reactor Summary for GiftKeeper Parent 1.0.0-SNAPSHOT:
[INFO]
[INFO] GiftKeeper Parent ..... SUCCESS [ 0.307 s]
[INFO] giftkeeper-domain ..... SUCCESS [ 9.705 s]
[INFO] giftkeeper-persistence-api ..... SUCCESS [ 0.327 s]
[INFO] giftkeeper-persistence-jpa ..... SUCCESS [ 27.747 s]
[INFO] giftkeeper-app ..... SUCCESS [ 10.800 s]
[INFO] giftkeeper-gui ..... SUCCESS [ 12.516 s]
[INFO] giftkeeper-cli ..... SUCCESS [ 7.911 s]
[INFO] giftkeeper-bdd ..... SUCCESS [ 5.843 s]
[INFO] giftkeeper-e2e ..... SUCCESS [ 22.795 s]
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 01:38 min
[INFO] Finished at: 2026-05-28T19:32:45+03:00
[INFO] -----
```

- GitHub Actions successful workflow

hussienghader / giftkeeper

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

actions [New workflow](#)

workflows

iftKeeper CI

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

All workflows

Showing runs from all workflows

Filter workflow runs

3 workflow runs

Workflow	Event	Status	Branch	Actor
Clean Eclipse warnings	GiftKeeper CI #19: Commit 6ac7307 pushed by hussienghader	main	36 minutes ago	1m 50s
Final reviewed academic submission cleanup	GiftKeeper CI #18: Commit e61e3be pushed by hussienghader	main	May 27, 5:59 PM GMT+3	1m 42s
Final Sonar cleanup	GiftKeeper CI #14: Commit e83de22 pushed by hussienghader	main	May 27, 3:29 PM GMT+3	1m 33s

- SonarCloud Quality Gate

hussienghader / giftkeeper

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

actions [New workflow](#)

workflows

iftKeeper CI

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

All workflows

Showing runs from all workflows

Filter workflow runs

3 workflow runs

Workflow	Event	Status	Branch	Actor
Clean Eclipse warnings	GiftKeeper CI #19: Commit 6ac7307 pushed by hussienghader	main	36 minutes ago	1m 50s
Final reviewed academic submission cleanup	GiftKeeper CI #18: Commit e61e3be pushed by hussienghader	main	May 27, 5:59 PM GMT+3	1m 42s
Final Sonar cleanup	GiftKeeper CI #14: Commit e83de22 pushed by hussienghader	main	May 27, 3:29 PM GMT+3	1m 33s

- PIT mutation testing report

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
7	100% 131/131	100% 77/77	100% 77/77

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.giftkeeper.domain	7	100% 131/131	100% 77/77	100% 77/77

Report generated by [PIT](#) 1.16.1

Enhanced functionality available at [arcmutate.com](#)

Pit Test Coverage Report

Package Summary

com.giftkeeper.domain

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
7	100% 131/131	100% 77/77	100% 77/77

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
Budget.java	100% 14/14	100% 12/12	100% 12/12
GiftIdea.java	100% 44/44	100% 25/25	100% 25/25
GiftStatus.java	100% 9/9	100% 5/5	100% 5/5
Occasion.java	100% 21/21	100% 11/11	100% 11/11
Person.java	100% 24/24	100% 11/11	100% 11/11
ReminderEngine.java	100% 7/7	100% 3/3	100% 3/3
ReminderPolicy.java	100% 12/12	100% 10/10	100% 10/10

Report generated by [PIT](#) 1.16.1

- GUI execution

[illegible]

- CLI execution output

```

May 28, 2026 8:53:22 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001001: Connection properties: {password=****, user=giftkeeper}
May 28, 2026 8:53:22 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl buildCreator
INFO: HHH10001003: Autocommit mode: false
May 28, 2026 8:53:22 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PooledConnect
INFO: HHH10001115: Connection pool size: 20 (min=1)
May 28, 2026 8:53:25 PM org.hibernate.engine.transaction.jta.platform.internal.JtaPlatformInitiator initiateService
INFO: HHH0000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
May 28, 2026 8:53:25 PM org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIso
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironment
[main] INFO com.giftkeeper.cli.GiftKeeperCliMain - 66ca6f80-d0b5-4aec-ad99-359a9f6c36e7 | alhasan Ghader | 1990-08-18
[main] INFO com.giftkeeper.cli.GiftKeeperCliMain - 88bce285-fe6e-4678-96b9-ef2735bef901 | Alhussein | 1998-06-08
[main] INFO com.giftkeeper.cli.GiftKeeperCliMain - 1020de10-74dc-4071-b698-ef7b6b8ea4b | AlhusseinGhader | 1998-06-08
[main] INFO com.giftkeeper.cli.GiftKeeperCliMain - dc5def77-65ea-4552-87c6-1eb58e6e2f3a | Luciana Bordiglia | 1961-08-18
[main] INFO com.giftkeeper.cli.GiftKeeperCliMain - 8f4c252b-4ba0-48c2-8a4e-c8e57f9d33fa5 | Luciana Bordiglia | 1961-08-18
[main] INFO com.giftkeeper.cli.GiftKeeperCliMain - db48b6c4-bd78-4b34-92d0-5f73f079e45ce | Zein Ghader | 1998-06-08

```